

PrepareCustomConfig#

https://pytorch.org/docs/stable/generated/torch.ao.quantization.fx.custom_config.PrepareCustomConfig.html

Description:

Custom configuration for `prepare_fx()` and `prepare_qat_fx()`. Create a `PrepareCustomConfig` from a dictionary with the following items:

- “standalone_module_name”: a list of (module_name, qconfig_mapping, example_inputs, child_prepare_custom_config, backend_config) tuples
- “standalone_module_class” a list of (module_class, qconfig_mapping, example_inputs, child_prepare_custom_config, backend_config) tuples

Function Signature

```
class
torch.ao.quantization.fx.custom_config.PrepareCustomConfig[source]#
classmethod from_dict(prepare_custom_config_dict)[source]#
Return type
set_float_to_observed_mapping(float_class, observed_class,
quant_type=QuantType.STATIC)[source]#
Return type
```

Returns:

dict[str, Any]

Code Examples

```
prepare_custom_config = PrepareCustomConfig()  
.set_standalone_module_name("module1", qconfig_mapping,  
example_inputs, child_prepare_custom_config,  
backend_config)  
.set_standalone_module_class(MyStandaloneModule,  
qconfig_mapping, example_inputs,  
child_prepare_custom_config, backend_config)  
.set_float_to_observed_mapping(FloatCustomModule,  
ObservedCustomModule)  
.set_non_traceable_module_names(["module2", "module3"])  
.set_non_traceable_module_classes([NonTraceableModule1,  
NonTraceableModule2])  
.set_input_quantized_indexes([0])  
.set_output_quantized_indexes([0])  
.set_preserved_attributes(["attr1", "attr2"])
```

Detailed Documentation

Documentation

Custom configuration for `prepare_fx()` and `prepare_qat_fx()`.

Create a `PrepareCustomConfig` from a dictionary with the following items:

“standalone_module_name”: a list of (module_name, qconfig_mapping, example_inputs, child_prepare_custom_config, backend_config) tuples

“standalone_module_class” a list of (module_class, qconfig_mapping, example_inputs, child_prepare_custom_config, backend_config) tuples

“float_to_observed_custom_module_class”: a nested dictionary mapping from quantization mode to an inner mapping from float module classes to observed module classes, e.g. {“static”: {FloatCustomModule: ObservedCustomModule}}

“non_traceable_module_name”: a list of modules names that are not symbolically traceable “non_traceable_module_class”: a list of module classes that are not symbolically traceable “input_quantized_idxes”: a list of indexes of graph inputs that should be quantized “output_quantized_idxes”: a list of indexes of graph outputs that should be quantized “preserved_attributes”: a list of attributes that persist even if they are not used in forward

This function is primarily for backward compatibility and may be removed in the future.

Set the mapping from a custom float module class to a custom observed module class.

The observed module class must have a from_float class method that converts the float module class to the observed module class. This is currently only supported for static quantization.

Set the indexes of the inputs of the graph that should be quantized. Inputs are otherwise assumed to be in fp32 by default instead.

Set the modules that are not symbolically traceable, identified by class.

Set the modules that are not symbolically traceable, identified by name.

Set the indexes of the outputs of the graph that should be quantized. Outputs are otherwise assumed to be in fp32 by default instead.

Set the names of the attributes that will persist in the graph module even if they are not used in the model's forward method.

Set the configuration for running a standalone module identified by `module_class`.

If `qconfig_mapping` is `None`, the parent `qconfig_mapping` will be used instead. If `prepare_custom_config` is `None`, an empty `PrepareCustomConfig` will be used. If `backend_config` is `None`, the parent `backend_config` will be used instead.

Set the configuration for running a standalone module identified by `module_name`.

If `qconfig_mapping` is `None`, the parent `qconfig_mapping` will be used instead. If `prepare_custom_config` is `None`, an empty `PrepareCustomConfig` will be used. If `backend_config` is `None`, the parent `backend_config` will be used instead.

Convert this `PrepareCustomConfig` to a dictionary with the items described in `from_dict()`.